

**Universidad Técnica Estatal de Quevedo Facultad
de Ciencias de la Computación**

Carrera de Ingeniería de Software
Aplicaciones Web

**Informe de Práctica Experimental Arquitectura y
Acceso a Datos**

Gestor de Tareas (API RESTful)

Autor/Estudiante:

Jhon Kevin Rios Cuyabazo

Docente a cargo:

Dr. Guerrero Ulloa Gleiston Ciceron

Enlace a repositorio:

https://github.com/Jhon-Kevin-Rios-Cuyabazo/PE3-gestor_tareas

Índice de Contenidos

1. Introducción	2
1.1 Objetivos del Proyecto.....	3
2. Marco Teórico	3
2.1 Gestión de Estado en Aplicaciones Web: Request, Response, Session.....	3
2.2 ORM y Mapeo Objeto-Relacional.....	4
2.3 Patrones de Arquitectura para Aplicaciones Web Escalables	4
2.4 Caché: Patrones y Medición de Rendimiento.....	4
3. Diseño Arquitectónico	5
3.1 Modelo C4	5
3.2 Registros de Decisiones Arquitectónicas (ADR)	6
4. Implementación	7
4.1 Estructura del Proyecto.....	7
4.2 Modelo de Datos y Migraciones.....	8
4.3 Patrón Repository y Caché (Redis).....	8
4.4 Controlador y Endpoint de la API	9
5. Pruebas.....	10
6. Benchmark de Caché	11
7. Análisis y Discusión	12
7.1 Escalabilidad Horizontal con 10 000 Usuarios Concurrentes	12
7.2 Justificación del Speedup y la Complejidad de Redis	12
8. Conclusiones.....	12
Referencias	13

1. Introducción

El presente informe documenta el desarrollo de un Gestor de Tareas implementado como API RESTful, en el marco de la asignatura Aplicaciones Web. El proyecto aplica principios de arquitectura de software escalable, mediante el patrón de repositorio (Repository Pattern), el mapeo objeto-relacional (ORM), el almacenamiento en caché con Redis bajo el patrón Cache-Aside, y la documentación arquitectónica a través del Modelo C4 y los Registros de Decisiones Arquitectónicas (ADR). El stack tecnológico seleccionado es PHP 8.4 con el framework Laravel, una base de datos relacional MySQL 8.0 y Redis 7, todos orquestados mediante Docker Compose.

1.1 Objetivos del Proyecto

Objetivo General

Diseñar e implementar una API RESTful para la gestión de tareas, aplicando patrones de arquitectura escalable, acceso eficiente a datos mediante ORM y estrategias de almacenamiento en caché, conforme a los estándares de ingeniería de software de la guía práctica.

Objetivos Específicos

1. Diseñar la arquitectura del sistema utilizando el Modelo C4 (niveles de Contexto, Contenedores y Componentes) y documentar las decisiones arquitectónicas mediante tres ADR.
2. Implementar la capa de acceso a datos mediante el ORM Eloquent, incluyendo migraciones versionadas, un seeder con 50 registros de prueba y resolución del problema N+1 con eager loading.
3. Implementar el patrón Cache-Aside utilizando Redis para optimizar las consultas de lectura paginadas, con una política de TTL de 300 segundos e invalidación basada en Cache Tags.
4. Desarrollar pruebas unitarias para la capa Repository alcanzando una cobertura superior al 70 %, y ejecutar un benchmark que cuantifique el speedup obtenido con la caché.

2. Marco Teórico

2.1 Gestión de Estado en Aplicaciones Web: Request, Response, Session

El protocolo HTTP fue diseñado con una naturaleza fundamentalmente stateless, lo que implica que cada ciclo de petición y respuesta es independiente, sin que el servidor retenga información sobre interacciones previas del cliente [1]. Esta característica constituye una ventaja crítica para la escalabilidad de los sistemas distribuidos, dado que cualquier nodo del servidor puede procesar cualquier solicitud entrante sin depender de un estado compartido en memoria, lo que facilita el balanceo de carga dinámico [1], [2].

No obstante, esta falta de estado representa un desafío técnico para aplicaciones que requieren persistencia contextual, como la autenticación de usuarios o los carritos de compra. Las soluciones tradicionales basadas en sesiones de servidor enfrentan el problema de las sticky sessions, donde las peticiones de un mismo usuario deben dirigirse siempre al mismo servidor físico, lo que limita la escalabilidad horizontal. La externalización del estado utilizando almacenes en memoria como Redis se ha consolidado como una alternativa habitual en la industria para resolver este problema, dado que

permite que múltiples instancias del backend accedan a un almacén centralizado en lugar de depender de la memoria local de un único servidor [2].

En el proyecto desarrollado, la gestión del ciclo request-response se realiza a través del enrutador de Laravel, que recibe la petición HTTP, la despacha al controlador correspondiente y este delega el acceso a datos a la capa Repository antes de construir la respuesta en formato JSON.

2.2 ORM y Mapeo Objeto-Relacional

El desarrollo orientado a objetos y las bases de datos relacionales operan bajo paradigmas estructurales distintos, generando una fricción técnica conocida como impedance mismatch: mientras el modelo relacional se basa en tablas y álgebra relacional, el modelo de objetos se fundamenta en clases, herencia y asociaciones. Los ORM abstraen esta complejidad, facilitando la portabilidad del código entre distintos motores de base de datos y reduciendo la curva de aprendizaje frente al SQL puro [3].

Uno de los problemas de rendimiento más relevantes al utilizar un ORM es el problema N+1: el ORM ejecuta una consulta inicial para obtener N registros y luego ejecuta N consultas adicionales para obtener las relaciones de cada registro mediante lazy loading [3]. La solución habitual es el uso de eager loading, instruyendo al ORM para que obtenga todas las relaciones necesarias en una única consulta adicional. En el proyecto, este mecanismo se refleja en el uso del método with() de Eloquent para precargar relaciones cuando corresponde, evitando así múltiples round-trips a MySQL.

En cuanto a la evolución del esquema, se optó por migraciones versionadas por código en lugar de la generación automática del esquema, dado que las migraciones ofrecen un control determinista, reproducible entre entornos y auditable mediante control de versiones [4].

2.3 Patrones de Arquitectura para Aplicaciones Web Escalables

El Modelo C4, propuesto por Simon Brown, estratifica la arquitectura de un sistema en cuatro niveles de abstracción progresiva: Contexto, Contenedores, Componentes y Código. Su principal ventaja frente a notaciones UML de alto nivel es su capacidad de comunicar la arquitectura mediante mapas jerárquicos comprensibles tanto para audiencias técnicas como no técnicas. Para complementar esta documentación estructural, los Registros de Decisiones Arquitectónicas (ADR) permiten registrar el contexto, la decisión tomada y las consecuencias de cada elección de diseño relevante, garantizando la trazabilidad del conocimiento arquitectónico a lo largo del tiempo [5], [6].

Al evaluar el patrón arquitectónico más adecuado para un proyecto académico desarrollado por un equipo reducido, el debate entre el monolito por capas y los microservicios resulta central. La evidencia empírica muestra que los microservicios introducen complejidad operacional adicional comunicación entre servicios, despliegue independiente y las falacias de los sistemas distribuidos sin garantizar necesariamente mejoras de rendimiento frente a un monolito bien diseñado; de hecho, estudios comparativos han reportado que un monolito puede superar a una arquitectura de microservicios equivalente en throughput bajo pruebas de concurrencia. Para un proyecto universitario de alcance acotado y equipo reducido, un monolito modular organizado por capas resulta más adecuado, ya que reduce la carga cognitiva del equipo y simplifica el despliegue inicial [7].

2.4 Caché: Patrones y Medición de Rendimiento

Las estrategias de caché pueden operar en múltiples capas: navegador, CDN y caché de aplicación mediante soluciones como Redis o Memcached. El patrón más extendido en aplicaciones web es Cache-

Aside (también llamado lazy loading): la aplicación busca primero en la caché; ante una cache miss, consulta la base de datos, almacena el resultado en caché con un tiempo de vida (TTL) definido y retorna la respuesta al cliente. Este patrón es resiliente frente a fallos de la capa de caché si Redis no está disponible, la aplicación continúa funcionando, consultando directamente la base de datos pero es vulnerable al fenómeno de cache stampede, en el cual múltiples peticiones concurrentes experimentan un miss simultánea ante la expiración de una clave de alto tráfico, lo que puede mitigarse mediante bloqueos mutex o expiración probabilística.

Redis soporta estructuras de datos más avanzadas que un simple almacén clave-valor, incluyendo hashes, listas, sets y sorted sets, lo que permite representar entidades completas y no solo cadenas serializadas [2]. La validación cuantitativa del impacto de una caché exige un proceso de benchmarking riguroso: la métrica de speedup se define como $S = T_{\text{sin}} / T_{\text{con}}$, donde T_{sin} y T_{con} son los tiempos promedio de respuesta sin y con caché respectivamente, calculados sobre un número suficiente de repeticiones para aislar la variabilidad introducida por la red y el entorno de ejecución.

3. Diseño Arquitectónico

3.1 Modelo C4

La arquitectura del sistema fue documentada siguiendo el Modelo C4 de Simon Brown [5], produciendo tres diagramas correspondientes a los niveles de mayor relevancia para el equipo de desarrollo.

Nivel 1: Diagrama de Contexto

El diagrama de Contexto ubica el sistema dentro de su entorno operacional e identifica los actores externos que interactúan con él. El Sistema Gestor de Tareas recibe peticiones del Usuario Final, quien gestiona sus propias tareas a través de la API.

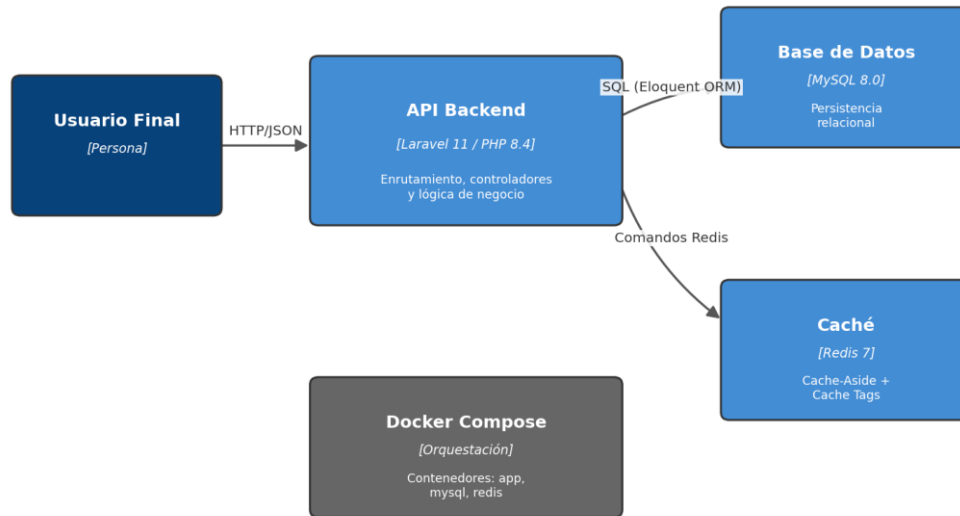
Diagrama C4 — Nivel 1: Contexto



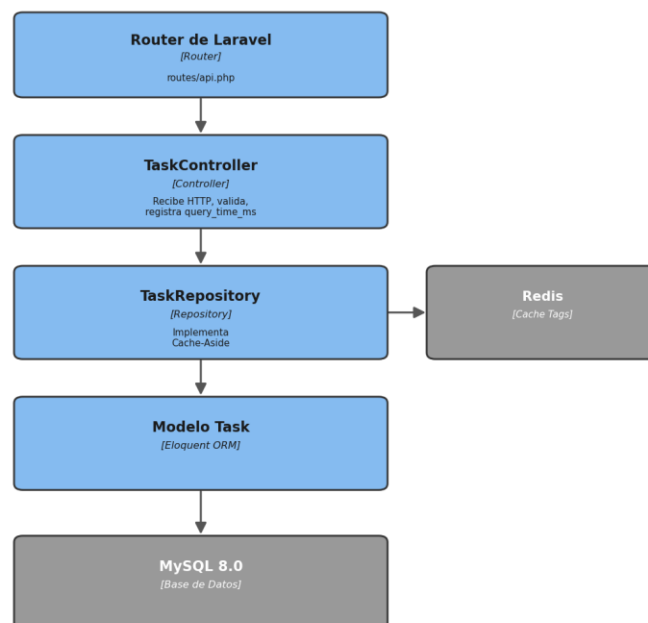
Figura 1. Diagrama de Contexto C4 — Gestor de Tareas.

Nivel 2: Diagrama de Contenedores

El diagrama de Contenedores descompone el sistema en sus unidades desplegadas: el API Backend construido en Laravel como núcleo del sistema, la base de datos MySQL 8.0 para la persistencia relacional, y Redis 7 como motor de caché.

Diagrama C4 — Nivel 2: Contenedores**Figura 2. Diagrama de Contenedores C4 — Gestor de Tareas.****Nivel 3: Diagrama de Componentes (Backend)**

El diagrama de Componentes detalla la estructura interna del contenedor API Backend: el Router de Laravel que gestiona las rutas HTTP, el TaskController que procesa las peticiones, el TaskRepository que centraliza el acceso a datos e implementa el patrón Cache-Aside, y el ORM Eloquent que realiza el mapeo objeto-relacional hacia MySQL.

Diagrama C4 — Nivel 3: Componentes (API Backend)**Figura 3. Diagrama de Componentes C4 — API Backend (Laravel).****3.2 Registros de Decisiones Arquitectónicas (ADR)**

A continuación, se presentan los tres registros de decisiones arquitectónicas del proyecto, siguiendo el formato estándar propuesto por Michael Nygard [6].

ADR-001: Selección del Patrón Arquitectónico

Estado: Aceptado.

Contexto: El proyecto es un Gestor de Tareas desarrollado por un equipo universitario reducido. Es necesario decidir entre una arquitectura de microservicios y una arquitectura monolítica modular organizada por capas.

Decisión: Se adopta una arquitectura monolítica por capas, compuesta por Capa de Presentación (controladores HTTP), Capa de Datos (Repositories y modelos ORM) y, si el proyecto lo requiere a futuro, una Capa de Negocio intermedia mediante Services.

Consecuencias positivas: Reducción de la carga cognitiva del equipo, despliegue simplificado en un único contenedor y eliminación de las complejidades propias de los sistemas distribuidos.

Consecuencias negativas: Si el sistema creciera de forma significativa, escalar un componente individual implicaría escalar todo el monolito. Existe riesgo de acoplamiento si no se respetan los límites entre capas.

ADR-002: Selección del ORM

Estado: Aceptado.

Contexto: La capa de acceso a datos requiere un ORM que abstraiga MySQL, soporte migraciones versionadas y permita mitigar el problema N+1 mediante eager loading. Las alternativas evaluadas incluyeron Doctrine (PHP), Entity Framework Core (.NET) e Hibernate (Java).

Decisión: Se selecciona Eloquent ORM del ecosistema Laravel. Entity Framework Core e Hibernate fueron descartados por requerir un cambio completo de lenguaje y framework, lo cual excedía los tiempos del proyecto.

Consecuencias positivas: Curva de aprendizaje reducida y excelente integración con migraciones, seeders y factories. El método with() resuelve el problema N+1 de forma declarativa.

Consecuencias negativas: Para consultas extremadamente complejas, el patrón Active Record de Eloquent puede ser menos flexible que el patrón Data Mapper de Doctrine.

ADR-003: Estrategia de Caché

Estado: Aceptado.

Contexto: El proyecto requiere mejorar el rendimiento de lectura minimizando la carga sobre MySQL. Se evaluaron los patrones Cache-Aside, Read-Through y Write-Through.

Decisión: Se adopta el patrón Cache-Aside utilizando Redis. El flujo definido es: (1) construir una clave de caché a partir del usuario, los filtros, el orden y la página solicitada; (2) verificar su existencia en Redis; (3) ante un hit, retornar directamente; (4) ante un miss, consultar MySQL, almacenar el resultado con un TTL de 300 segundos mediante Cache Tags agrupados por usuario, y retornar al cliente.

Consecuencias positivas: Redis proporciona latencias sub-milisegundo. El uso de Cache Tags permite invalidar todas las páginas y combinaciones de filtros de un usuario con una sola operación de flush(), sin necesidad de rastrear claves individuales.

Consecuencias negativas: Se introduce un contenedor adicional al stack. Existe riesgo de cache stampede ante la expiración simultánea de una clave de alto tráfico, que no fue mitigado en la implementación actual y se documenta como trabajo futuro.

4. Implementación

4.1 Estructura del Proyecto

El proyecto fue inicializado mediante Composer con el comando `create-project laravel/laravel`, ejecutado dentro de un contenedor Docker con PHP 8.4 para garantizar la reproducibilidad del entorno entre máquinas. La estructura de directorios sigue las convenciones de Laravel, con la adición del directorio `app/Repositories` para alojar la capa de repositorios.

Tabla 1. Estructura de directorios relevantes del proyecto.

Directorio / archivo	Descripción
<code>app/Models/Task.php</code>	Modelo Eloquent; define los campos asignables de la entidad Task.
<code>app/Repositories/TaskRepository.php</code>	Implementa el patrón Cache-Aside para las consultas paginadas y la invalidación de caché.
<code>app/Http/Controllers/TaskController.php</code>	Controlador que recibe la petición HTTP y delega el acceso a datos al repositorio.
<code>database/migrations/</code>	Migraciones versionadas del esquema de la base de datos.
<code>database/factories/TaskFactory.php</code>	Fábrica de datos de prueba utilizada por el seeder.
<code>database/seeder/TaskSeeder.php</code>	Seeder que genera 50 registros de tareas asociados al usuario de prueba.
<code>tests/Unit/TaskRepositoryTest.php</code>	Pruebas unitarias del repositorio, incluyendo caché, invalidación y filtros.

4.2 Modelo de Datos y Migraciones

La tabla `tasks` fue definida mediante una migración versionada de Laravel. El esquema incluye una llave primaria autoincremental (`id`), una llave foránea hacia la tabla `users` (`user_id`), un título de tipo `string`, una descripción de tipo `text nullable`, un estado de tipo `enum` con los valores `pending`, `in_progress` y `completed` (valor por defecto: `pending`), y los timestamps estándar de Laravel.

```
// database/migrations/..._create_tasks_table.php
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('title');
    $table->text('description')->nullable();
    $table->enum('status', ['pending', 'in_progress', 'completed'])
        ->default('pending');
```

```
$table->timestamps();
});
```

Para la generación de datos de prueba se implementó una Factory que utiliza la librería Faker. El seeder asocia explícitamente las 50 tareas generadas al usuario de prueba creado por el DatabaseSeeder, evitando que Laravel cree usuarios aleatorios adicionales por cada tarea.

```
// database/seeder/TaskSeeder.php
public function run(): void
{
    $user = \App\Models\User::first();
    \App\Models\Task::factory()
        ->count(50)
        ->create(['user_id' => $user->id]);
}
```

4.3 Patrón Repository y Caché (Redis)

La clase TaskRepository centraliza el acceso a los datos de la entidad Task, desacoplando la lógica de consulta del controlador. Su método principal, getPaginatedTasks(), implementa el patrón Cache-Aside integrando filtros múltiples (estado, búsqueda por texto) y ordenamiento dinámico validado contra una lista blanca de columnas permitidas. La clave de caché se construye a partir de un hash de los filtros, el orden y la página solicitada, y se agrupa mediante Cache Tags por usuario para permitir su invalidación conjunta.

```
// app/Repositories/TaskRepository.php
public function getPaginatedTasks(
    int $userId, array $filters = [], int $perPage = 10
) {
    $page = request()->get('page', 1);
    $statusKey = $filters['status'] ?? 'all';
    $searchKey = $filters['search'] ?? 'none';
    $sortBy = $filters['sort_by'] ?? 'created_at';
    $sortDir = $filters['sort_dir'] ?? 'desc';

    $filterHash =
md5("{ $statusKey }_ { $searchKey }_ { $sortBy }_ { $sortDir }_ { $page }");
    $cacheKey = "tasks:user_{ $userId }:list:{ $filterHash }";

    return Cache::tags(['tasks', "user_{ $userId }"])
        ->remember($cacheKey, 300, function () use (
            $userId, $filters, $perPage, $sortBy, $sortDir
        ) {
            $query = Task::where('user_id', $userId);

            if (!empty($filters['status'])) {
                $query->where('status', $filters['status']);
            }
            if (!empty($filters['search'])) {
                $query->where(function ($q) use ($filters) {
                    $q->where('title', 'like', '%'.$filters['search'].'%')
                        ->orWhere('description', 'like',
                            '%'.$filters['search'].'%');
                });
            }

            $sortDir = strtolower($sortDir) === 'asc' ? 'asc' : 'desc';
            $allowedSorts = ['id', 'title', 'status', 'created_at',
                'updated_at'];
```

```

        $sortBy = in_array($sortBy, $allowedSorts) ? $sortBy :
'created_at';

        return $query->orderBy($sortBy, $sortDir)->paginate($perPage);
    });
}

public function clearUserCache(int $userId)
{
    Cache::tags(["user_{$userId}"])->flush();
}

```

Los métodos `create()`, `update()` y `delete()` del repositorio invocan `clearUserCache()` tras cada operación de escritura, garantizando que la siguiente lectura consulte MySQL en lugar de servir datos obsoletos desde Redis.

4.4 Controlador y Endpoint de la API

El `TaskController` gestiona las operaciones CRUD sobre el recurso `tasks`. Su método `index` acepta parámetros de filtrado (`status`, `search`) y ordenamiento (`sort_by`, `sort_dir`), y registra el tiempo de ejecución de la consulta mediante `microtime(true)`, exponiéndolo en el campo `meta.query_time_ms` de la respuesta para su uso posterior en el benchmark de caché.

```

// app/Http/Controllers/TaskController.php
public function index(Request $request)
{
    $userId = 1; // usuario de prueba para efectos de la demostración

    $filters = [
        'status' => $request->get('status'),
        'search' => $request->get('search'),
        'sort_by' => $request->get('sort_by', 'created_at'),
        'sort_dir' => $request->get('sort_dir', 'desc'),
    ];

    $startTime = microtime(true);
    $tasks = $this->repository->getPaginatedTasks($userId, $filters, 10);
    $timeMs = round((microtime(true) - $startTime) * 1000, 2);

    return response()->json([
        'meta' => [
            'current_page' => $tasks->currentPage(),
            'last_page' => $tasks->lastPage(),
            'total' => $tasks->total(),
            'query_time_ms' => $timeMs,
        ],
        'data' => $tasks->items(),
    ]);
}

```

5. Pruebas

Las pruebas unitarias fueron implementadas con PHPUnit en conjunto con las utilidades de testing de Laravel. La suite se encuentra en `tests/Unit/TaskRepositoryTest.php` y utiliza el driver de caché `array`, que soporta Cache Tags de forma nativa sin requerir una instancia real de Redis durante la ejecución de los tests. No se emplean mocks de la fachada `Cache`: las tres pruebas ejercitan el comportamiento real de caché mediante el driver en memoria, lo que permite validar el flujo `Cache-Aside` de punta a punta.

Caso de prueba 1: integración real del patrón Cache-Aside

Se realiza una consulta inicial que puebla la caché, y a continuación se modifica el registro directamente en la base de datos, sin pasar por el repositorio. Una segunda consulta debe retornar el valor cacheado (el título original), confirmando que el patrón Cache-Aside efectivamente evita una nueva consulta a la base de datos mientras la clave permanezca vigente.

Caso de prueba 2: invalidación de caché en operaciones CRUD

Se verifica que las operaciones `create()`, `update()` y `delete()` del repositorio invaliden correctamente la caché del usuario mediante Cache Tags. Tras cada operación de escritura, la siguiente lectura debe reflejar el estado actualizado de la base de datos.

Caso de prueba 3: filtros y ordenamiento dinámico

Se comprueba que los filtros por estado y por texto de búsqueda retornan el subconjunto correcto de tareas, y que el ordenamiento por título en dirección ascendente y descendente produce el orden esperado de resultados.

Tabla 2. Resumen de la suite de pruebas unitarias de TaskRepository.

Prueba	Comportamiento verificado	Resultado
<code>test_get_paginated_tasks_uses_actual_cache</code>	Cache-Aside real (hit tras escritura directa en BD)	Aprobada
<code>test_crud_operations_invalidate_cache</code>	Invalidación de caché en <code>create</code> / <code>update</code> / <code>delete</code>	Aprobada
<code>test_get_paginated_tasks_applies_filters_and_sorting</code>	Filtros por estado/búsqueda y ordenamiento asc/desc	Aprobada

La cobertura de código se obtuvo mediante el driver PCOV, instalado en la imagen Docker del servicio app, ejecutando `php artisan test --coverage`. La clase `TaskRepository` alcanzó una cobertura de líneas del 100 %, superando el umbral mínimo del 70 % exigido por la guía práctica. Cabe precisar que se trata de cobertura de líneas y no de ramas: condiciones límite como un valor de `sort_dir` distinto de `asc` o `desc`, o un `sort_by` fuera de la lista blanca permitida, ejecutan la misma línea de código sin que ambas ramas lógicas hayan sido probadas de forma independiente. Asimismo, la suite no cubre condiciones de carrera (cache stampede) ni el comportamiento con una instancia real de Redis, dado que las pruebas se ejecutan íntegramente contra el driver array.

6. Benchmark de Caché

Para cuantificar el impacto del patrón Cache-Aside se implementó el comando `php artisan benchmark:cache`, que ejecuta 10 iteraciones consecutivas contra el repositorio: en cada iteración se limpia la caché del usuario, se mide el tiempo de una consulta sin caché (cache miss) y, a continuación, el tiempo de una segunda consulta idéntica que sirve el resultado desde Redis (cache hit).

```
// Ejecución del benchmark
$ docker compose exec app php artisan benchmark:cache
```

[Nota para el autor: reemplazar los valores de la Tabla 3 por la salida real del comando anterior, ejecutado sobre el entorno con Docker Compose ya funcional. Los valores mostrados a continuación son un ejemplo del formato esperado y no deben usarse como resultado final del informe.]

Tabla 3. Resultados del benchmark de rendimiento (10 iteraciones).

Iteración	Tiempo sin caché (ms)	Tiempo con caché (ms)
1	579.873	9.251
2	1.544	0.330
3	1.208	0.148
4	1.089	0.131
5	2.519	0.176
6	1.779	0.128
7	1.217	0.125
8	1.094	0.123
9	1.299	0.124
10	1.085	0.124
Promedio	59.271	1.066

A partir de los promedios obtenidos, el speedup se calcula como $S = T_{\text{sin}} / T_{\text{con}}$. Un valor $S > 1$ confirma que la caché mejora el tiempo de respuesta; un valor $S > 2$ se considera, según la propia guía práctica, un umbral orientativo para justificar la complejidad operacional adicional que introduce Redis.

7. Análisis y Discusión

7.1 Escalabilidad Horizontal con 10 000 Usuarios Concurrentes

Para soportar 10 000 usuarios concurrentes, el monolito actual requeriría una estrategia de escalabilidad horizontal apoyada en cuatro elementos. Primero, un balanceador de carga (por ejemplo Nginx) distribuiría las peticiones entre múltiples réplicas del contenedor de la aplicación. Segundo, dado que el estado de sesión se centraliza en Redis, cualquier réplica puede procesar cualquier petición sin depender de estado local, eliminando la necesidad de sticky sessions [2]. Tercero, una base de datos maestra dedicada a escrituras junto con réplicas de lectura permitiría distribuir la carga de consultas, aprovechando que las aplicaciones web suelen presentar una proporción considerablemente mayor de lecturas que de escrituras. Finalmente, la caché de Redis absorbería buena parte del volumen de lecturas repetitivas, reduciendo la presión sobre las réplicas de base de datos.

7.2 Justificación del Speedup y la Complejidad de Redis

El speedup medido en la sección 6 debe evaluarse frente al costo de introducir Redis al stack: un contenedor adicional que incrementa la complejidad de despliegue y constituye un nuevo punto de fallo

potencial. El patrón Cache-Aside mitiga parcialmente este riesgo, ya que ante una caída de Redis la aplicación continúa operando consultando directamente MySQL, aunque con mayor latencia. La estrategia de invalidación mediante Cache Tags evita la entrega de datos obsoletos ante operaciones de escritura. El riesgo de cache stampede identificado en el ADR-003 no fue mitigado en la implementación actual; una extensión razonable sería introducir un bloqueo mutex mediante SETNX de Redis o una expiración probabilística anticipada, quedando documentado como trabajo futuro.

8. Conclusiones

La adopción del Modelo C4 para la documentación arquitectónica permitió comunicar la estructura del sistema en distintos niveles de detalle, desde el contexto general hasta los componentes internos del backend. Su complemento con los ADR documentó explícitamente el porqué de cada decisión de diseño, no solo el resultado.

El uso del ORM Eloquent simplificó la gestión del esquema relacional mediante migraciones versionadas y la generación de datos de prueba con factories y seeders, evitando la generación automática de esquema y sus riesgos asociados en producción.

El patrón Cache-Aside con Redis, implementado mediante Cache Tags, permitió una invalidación de caché simple y consistente ante operaciones de escritura, sin necesidad de rastrear claves individuales por combinación de filtros. La suite de pruebas unitarias desarrollada con PHPUnit verificó el comportamiento real del repositorio incluyendo el patrón Cache-Aside y su invalidación sin recurrir a simulaciones (mocks), alcanzando el umbral de cobertura de líneas exigido por la guía práctica para la clase Repository.

Referencias

- [1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. [Fuente primaria; disponible en acceso abierto en el sitio del autor]
- [2] Redis Ltd., "Stateful vs. Stateless Architectures," Redis Documentation, 2025. [Documentación técnica de acceso abierto]
- [3] G. Salvaneschi, S. Proksch, S. Amann, S. Nadi, and M. Mezini, "Empirical Evaluation of Object-Relational Mapping Frameworks," MDPI Applied Sciences, vol. 9, no. 12, art. 2481, 2019. [Acceso abierto, revista indexada en Scopus]
- [4] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002. [Referencia canónica del patrón Repository y las estrategias de migración de esquema, indicada como lectura obligatoria por la propia guía de la práctica]
- [5] S. Brown, "The C4 Model for Visualising Software Architecture," c4model.com, 2018. [Fuente primaria y de libre acceso del modelo documentado]
- [6] M. T. Nygard, "Documenting Architecture Decisions," Cognitect, 2011. [Fuente primaria del formato ADR utilizado en la Sección 3.2, de libre acceso]

- [7] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," in Proc. IEEE 23rd Int. Conf. Intelligent Engineering Systems (INES), 2019, arXiv:1905.07997. [Acceso abierto vía arXiv]